

ySpaceOS — YSPACE LOGISTICS PLATFORM

Master Product Requirements Document

Product Name

Yspace Logistics Platform

Operating System

ySpaceOS

Core Product

Yspace Drone Delivery + Autonomous Route Intelligence

Primary Interfaces

Web Application + Mobile Application + Enterprise Web Workspace + APIs/SDKs

Document Type

Product Requirements Document — Product, UX/UI, Engineering, AI, Data and Integration Specification

1. DOCUMENT PURPOSE

This document defines the complete product requirements for the **Yspace Logistics Platform**, an external-facing component of **ySpaceOS**, Yspace's broader operating system and intelligence infrastructure for autonomous logistics.

The purpose of this document is not simply to describe a website.

It is intended to provide a sufficiently detailed product specification for:

- Product design
- UI/UX design
- Frontend engineering
- Backend engineering

- AI/ML engineering
- Geospatial engineering
- API engineering
- Data engineering
- Enterprise integration
- Business development
- Operations
- Product management
- Future investor communication

The document should allow a UI/UX designer to understand:

- what screens need to exist;
- what each screen is supposed to accomplish;
- who uses each screen;
- what information appears on each screen;
- what actions are available;
- how users move between screens;
- what information is visible to customers;
- what information is visible to enterprise customers;
- what information should remain internal to Yspace;
- how the map should behave;
- how routing intelligence should be presented;
- how enterprise integrations work;
- how the customer-facing drone-delivery experience differs from the enterprise intelligence experience.

The product must therefore be designed as a **real logistics infrastructure product**, not merely as a delivery website.

2. PRODUCT DEFINITION

2.1 What is Yspace?

Yspace is a logistics and transportation company focused on using autonomous systems, software and intelligence to make transportation faster, more efficient and more intelligent.

The company's operational technology is built around **ySpaceOS**.

ySpaceOS is the underlying operating system and intelligence layer that can coordinate the digital systems required for Yspace's autonomous logistics operations.

One of the interfaces powered by ySpaceOS is **Yspace Mission Control**.

Mission Control is Yspace's internal logistics control tower.

It is used by Yspace personnel to operate and monitor Yspace's own autonomous transportation operations, particularly its drone fleet.

The **Yspace Logistics Platform** is different.

It is the external commercial platform through which customers and businesses interact with Yspace.

3. THE TWO CORE COMMERCIAL USE CASES

The Yspace Logistics Platform has two major commercial functions.

They should remain clearly separated in the product even though they are powered by the same broader ySpaceOS infrastructure.

3.1 Use Case A — Yspace Drone Delivery

Yspace operates the physical delivery service.

A customer can:

- visit the Yspace website;
- open the Yspace application;
- provide a pickup location;
- provide a destination;
- provide package information;
- receive a delivery estimate;
- pay;
- hand over the package to Yspace;
- track the delivery;
- receive the package at its destination.

Yspace is responsible for the operational execution.

The customer does not operate the drone.

The customer does not select the aircraft.

The customer does not plan the flight path.

The customer does not control the drone.

The customer does not access raw telemetry.

Those responsibilities belong to Yspace's internal operational infrastructure, including Mission Control.

3.2 Use Case B — Autonomous Route Intelligence for Businesses

The second product is fundamentally different.

This product is designed for:

- logistics companies;
- e-commerce companies;
- marketplaces;
- retailers;
- delivery companies;
- transportation companies;
- businesses with their own delivery operations;
- enterprises operating motorcycles;
- enterprises operating cars;
- enterprises operating vans;
- enterprises operating trucks;
- businesses with their own fleet-management software.

These companies are **not asking Yspace to manage their riders or vehicles.**

They already have their own logistics operation.

They may already have:

- fleet-management software;
- dispatch software;
- rider applications;
- driver applications;
- ERP systems;
- order-management systems;
- warehouse systems;

- delivery-management systems.

What they need from Yspace is intelligence.

The core proposition is therefore:

Yspace provides the intelligence that helps a business determine where its vehicles should travel and how they should travel there more efficiently.

The enterprise's own system remains responsible for managing its operations.

4. THE MOST IMPORTANT PRODUCT DISTINCTION

This distinction must be preserved throughout the entire product.

Yspace Drone Delivery

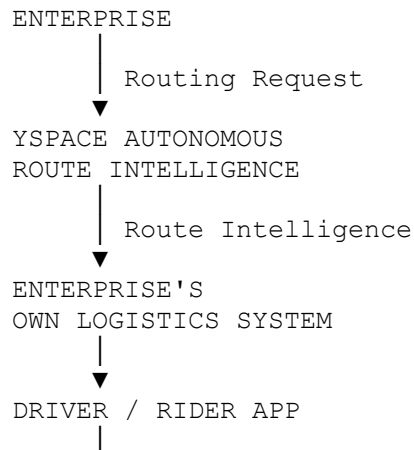
Yspace operates the transportation.

Enterprise Autonomous Route Intelligence

The enterprise operates the transportation. Yspace provides intelligence.

This means Yspace does not need to become the fleet-management platform for every company using Autonomous Route Intelligence.

Instead:





VEHICLE

This architecture is central to the product.

5. WHAT THE ENTERPRISE IS ACTUALLY BUYING

The enterprise is not buying:

- rider management;
- driver management;
- vehicle management;
- dispatch management;
- Yspace fleet operations.

The enterprise is buying access to **Yspace's logistics intelligence**.

That intelligence can include:

- route calculation;
- route optimization;
- alternative route generation;
- traffic-aware routing;
- estimated travel time;
- ETA prediction;
- route comparison;
- route scoring;
- route constraints;
- vehicle-specific routing profiles;
- multi-stop routing;
- route history;
- route analytics;
- route reliability;
- API access;
- SDK integration;
- predictive route intelligence;
- logistics risk intelligence.

6. WHY THIS PRODUCT EXISTS

Basic digital maps can already tell someone how to get from point A to point B.

That alone is not the Yspace product.

Modern mapping providers already expose sophisticated routing capabilities. For example, Mapbox provides directions, traffic-aware routing, matrices, map matching and route optimization services. Its Matrix API can return travel times between many points, which can be used as an input to optimization algorithms.

Google's Route Optimization API similarly allows businesses to optimize routes according to objectives and constraints including travel efficiency, time windows, vehicle capacity and other operational requirements.

Therefore, Yspace must not position its product merely as:

"We have a routing algorithm."

That would be too narrow.

The Yspace opportunity is to build a **logistics intelligence layer** that combines routing technology with enterprise-specific data, local conditions, operational context, ETA intelligence, historical performance, route explanations and integrations.

The long-term product should answer not only:

"What route can this vehicle take?"

but:

"What is the best transportation decision given what we know about this road network, this vehicle, this business, this time, this destination and current conditions?"

7. PRODUCT PHILOSOPHY

The product should follow six principles.

Principle 1 — Intelligence, not unnecessary complexity

The user should receive useful decisions without needing to understand the mathematics behind them.

Principle 2 — Enterprise-owned operations remain enterprise-owned

Yspace provides intelligence.

The enterprise remains responsible for its own:

- drivers;
- riders;
- vehicles;
- fleet;
- dispatch;
- operational execution.

Principle 3 — The API is a first-class product

Enterprise customers should not be forced to manually open Yspace every time they need a route.

They should eventually be able to integrate Yspace directly into their existing systems.

Principle 4 — Human-readable intelligence

The system should explain recommendations.

Instead of:

Route A selected.

It should say:

Route A recommended — approximately 9 minutes faster under current conditions.

Principle 5 — Separate internal and external information

Mission Control may contain highly detailed operational information.

The customer platform should only expose information necessary for the customer's experience.

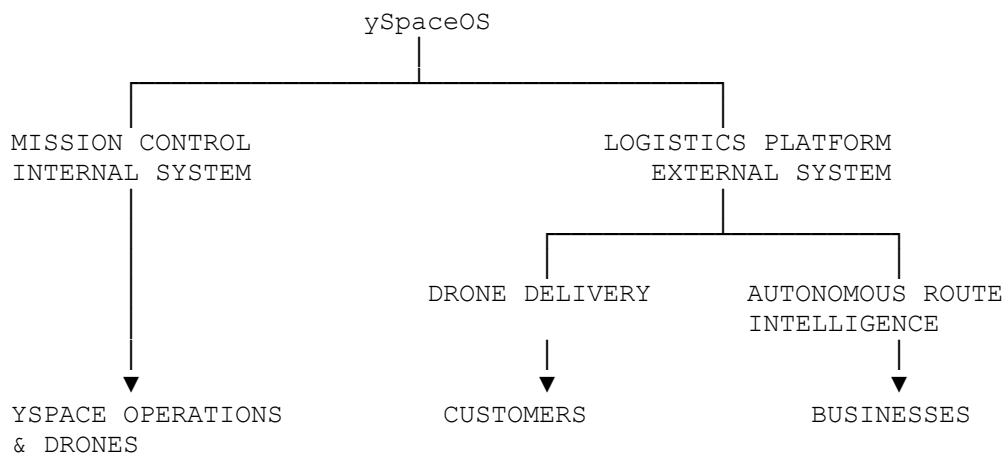
Principle 6 — Build the intelligence layer progressively

The initial product should not pretend to have predictive intelligence that has not been trained or validated.

The architecture should be designed to support increasingly sophisticated intelligence as Yspace collects data.

8. RELATIONSHIP TO ySpaceOS

The conceptual hierarchy is:



The two external products use shared infrastructure where appropriate.

9. MISSION CONTROL VS LOGISTICS PLATFORM

Mission Control

Mission Control is internal.

It exists to operate Yspace's own logistics network.

It may contain:

- drone telemetry;
- drone status;
- mission status;
- autonomous mission planning;
- flight routes;
- airspace information;
- battery state;
- operational alerts;
- aircraft health;
- mission risk;
- internal operational decisions.

Customers should not see this internal layer.

Logistics Platform

The Logistics Platform is external.

It exists to:

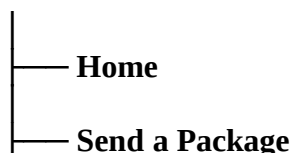
1. sell Yspace's delivery service;
2. provide customers with delivery visibility;
3. sell Yspace's routing intelligence to businesses.

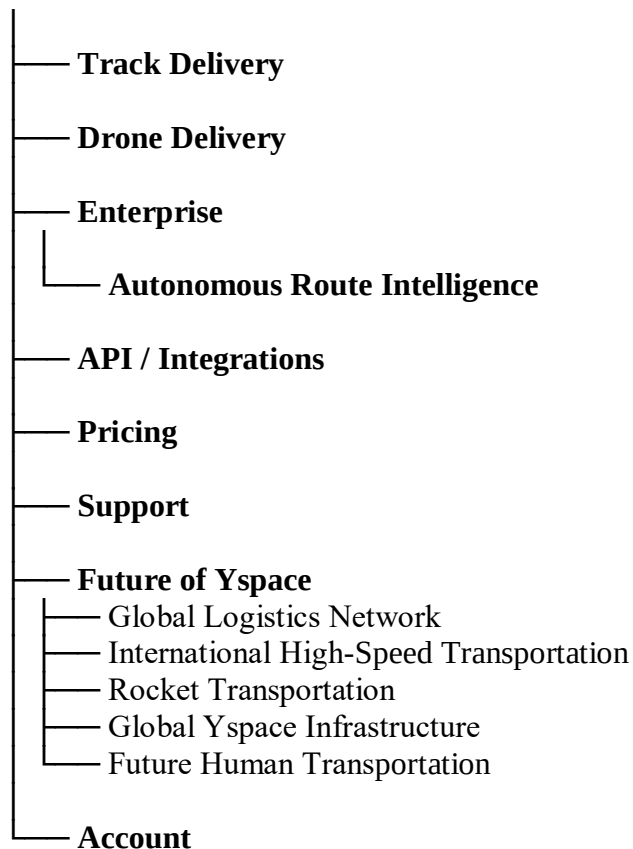
10. PRODUCT INFORMATION ARCHITECTURE

The public Yspace platform should clearly separate **current services, business products, operational technologies, and Yspace's long-term future vision.**

The primary structure should be:

YSPACE





Future of Yspace

The **Future of Yspace** section should be treated differently from the company's currently available products.

It represents Yspace's **long-term vision and planned capabilities**, rather than services that users can currently book.

This page can explain how Yspace intends to evolve from an autonomous logistics and delivery company into a broader transportation infrastructure company.

The vision includes:

- Building Yspace logistics hubs across strategic countries and major markets.
- Connecting international transportation with Yspace's local delivery network.
- Eventually using next-generation high-speed transportation, including rocket-based transportation, for international movement of goods.
- Moving packages from international origins directly into Yspace's local autonomous delivery network.
- Eventually developing transportation capabilities for humans, subject to technological development, regulation, certification and safety requirements.

The public website should therefore make a clear distinction between:

AVAILABLE TODAY

What customers and businesses can actually use now.

THE FUTURE

What Yspace intends to build over time.

The enterprise experience should not be mixed into the ordinary consumer booking flow.

Likewise, future transportation concepts should not be presented as currently available booking services. They should be presented as part of the **Yspace vision and roadmap**.

This structure allows Yspace to communicate an extremely ambitious long-term vision while maintaining complete clarity about what the company actually offers today..

11. THE HOME PAGE

The home page must immediately communicate the two things Yspace does.

Primary message

Move what matters, faster.

Supporting statement:

Yspace combines autonomous delivery with intelligent logistics technology to make transportation faster and more efficient.

Two major CTAs:

Send a Package

For customers who want Yspace to deliver something.

For Businesses

For companies interested in Yspace Autonomous Route Intelligence.

11.1 THE FUTURE OF YSPACE

The homepage should also introduce Yspace's long-term vision without confusing it with the services currently available.

Near the bottom of the homepage, Yspace should feature a dedicated **future vision section** that communicates where the company is ultimately heading.

The message should be ambitious but clearly positioned as a **future capability**, not a service currently available.

Future Vision

From local delivery to global transportation.

Yspace's long-term vision is to build an end-to-end transportation network capable of moving packages — and eventually people — between countries in dramatically shorter times.

Instead of relying entirely on traditional freight infrastructure and conventional air transportation, Yspace plans to eventually integrate **next-generation high-speed transportation technologies, including rocket-based transportation**, into its logistics network.

For example:

China



Yspace International Hub



High-Speed / Rocket Transportation



Nigeria



Yspace Local Hub



Autonomous Delivery



Customer

The long-term objective is for Yspace to control more of the journey rather than simply handing a package from one transportation provider to another.

Yspace could establish operational hubs and offices across strategic countries and major logistics markets, connecting international transportation infrastructure with its local autonomous delivery network.

A package travelling from China to Nigeria could eventually move through the Yspace network from international origin, through high-speed transportation, into a Nigerian Yspace hub, and finally to the customer's doorstep through autonomous delivery.

This represents the broader Yspace vision:

International transportation + logistics infrastructure + autonomous delivery = one connected transportation network.

The same infrastructure could eventually support **human transportation**, subject to the required technology, safety standards, regulation and certification.

Homepage Presentation

This section should be visually distinct from the current services and explicitly communicate that it represents the future.

Suggested headline:

The future of transportation is faster.

Supporting copy:

Today, we move packages across cities. Tomorrow, we intend to connect the world — combining intelligent logistics, autonomous delivery and next-generation transportation to move packages and eventually people across borders in dramatically less time.

Primary CTA:

Explore Our Vision →

Secondary CTA:

Learn More →

The CTA should lead to a dedicated **Yspace Future / Vision** page where users can explore the company's long-term plans, including international transportation, global infrastructure, autonomous logistics, rocket-based transportation and eventually human transportation.

The homepage should therefore communicate a clear progression:

Today:

Autonomous and intelligent local logistics.

Next:

Expanded autonomous delivery networks and international logistics infrastructure.

Future:

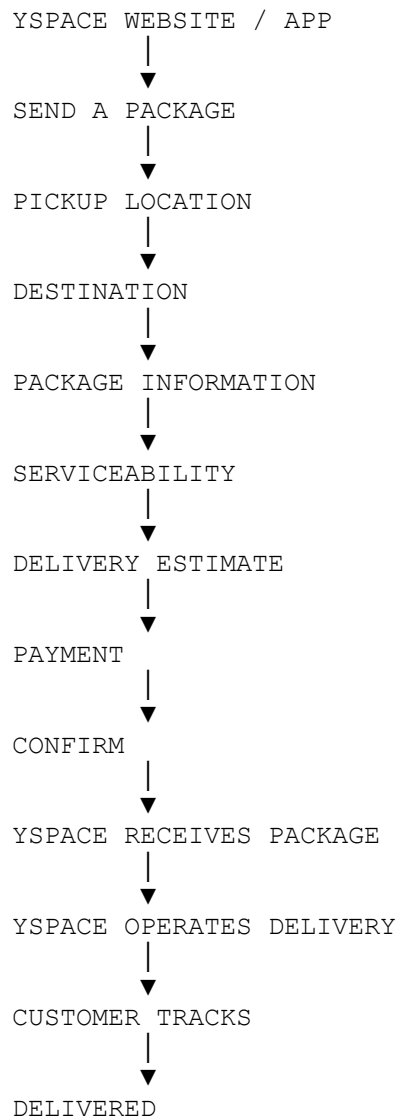
High-speed international transportation integrated directly with Yspace's logistics network.

Long-term:

A globally connected transportation ecosystem capable of moving both **goods and people**.

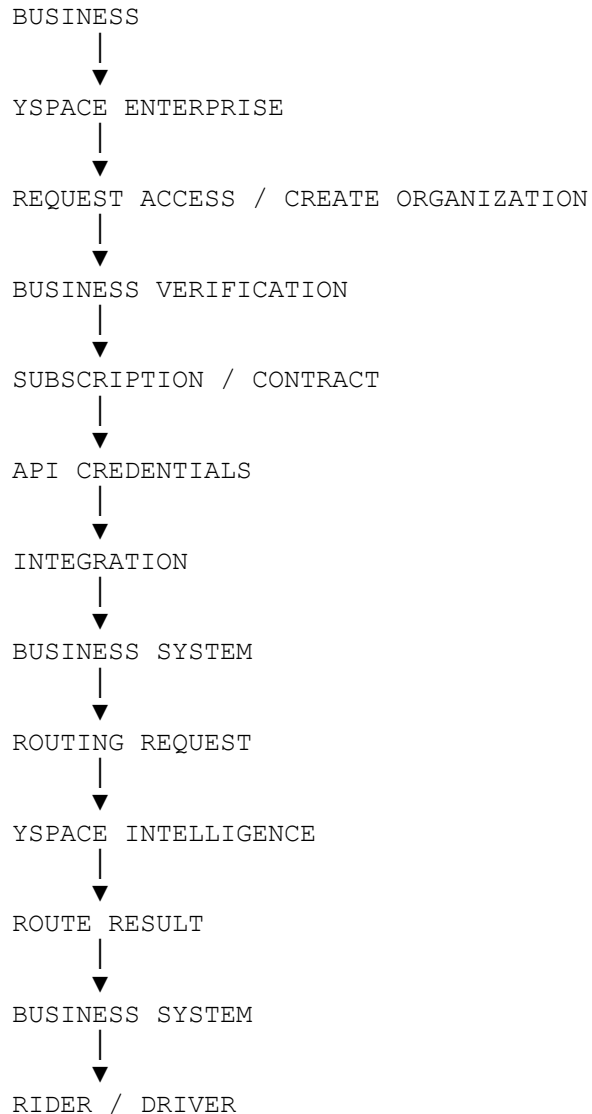
12. CUSTOMER FLOW

The normal customer journey is:



13. BUSINESS FLOW

The enterprise journey is completely different.



This distinction should be obvious in the UI.

14. ENTERPRISE IS AN ORGANIZATION

A company should not behave like an ordinary customer account.

The platform should support an **Organization** model.

Example:

Organization:
ABC Logistics Ltd.

Organization ID:
ORG_00192

Subscription:
Enterprise

Users:
12

API Keys:
4

Routing Profiles:
3

The organization owns the subscription.

Users are members of the organization.

The API belongs to the organization.

15. ENTERPRISE USER ROLES

Recommended roles:

Organization Owner

Full organizational control.

Can:

- manage subscription;
- manage billing;
- manage users;
- manage API credentials;
- configure routing profiles;
- view analytics.

Administrator

Can:

- manage users;
- configure routing;
- view analytics;
- manage integrations.

Operations Analyst

Can:

- analyze routes;
- compare routes;
- view route history;
- configure routing parameters.

Developer

Can:

- create API keys;
- manage integrations;
- configure webhooks;
- test API requests;
- view API usage.

Billing Manager

Can:

- view invoices;
- manage payment;
- view usage;
- manage subscription.

Viewer

Read-only access.

16. ENTERPRISE ACCOUNT CREATION

The enterprise flow should not be:

Create normal customer account → enter package weight → book delivery.

Instead:

```
FOR BUSINESSES
↓
GET STARTED
↓
CREATE BUSINESS ACCOUNT
↓
BUSINESS INFORMATION
↓
ORGANIZATION CREATED
↓
CHOOSE AUTONOMOUS ROUTE INTELLIGENCE
↓
SUBSCRIPTION / SALES PROCESS
↓
WORKSPACE
```

17. ENTERPRISE BUSINESS INFORMATION

The system may request:

- company name;
- business type;
- country;
- operating locations;
- estimated routing volume;
- vehicle types;
- number of vehicles;
- current logistics technology;
- integration requirements.

The purpose is not to manage the fleet.

The purpose is to understand the routing problem.

18. ENTERPRISE ONBOARDING

The onboarding should ask:

What are you trying to improve?

Options:

- Faster routes
- Better ETA
- Multi-stop optimization
- Traffic-aware routing
- Route analytics
- API integration
- Other

Then:

What vehicles do you operate?

- Motorcycles
- Cars
- Vans
- Trucks
- Other

Again, this is **routing context**, not fleet management.

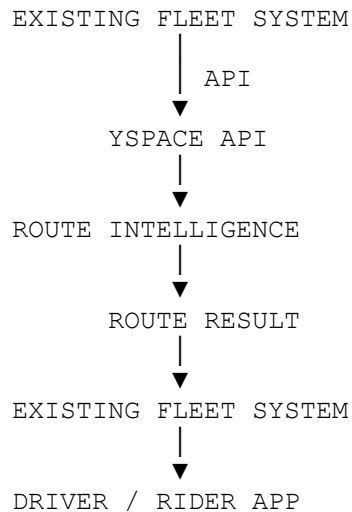
19. EXISTING FLEET SYSTEM INTEGRATION

This is critical.

A logistics company may already have a fleet-management system.

Yspace should not ask the company to abandon it.

Instead:



This is one of the most important architectural decisions in the product.

20. WHY INTEGRATION MATTERS

Suppose a logistics company has 5,000 motorcycles.

It already knows:

- where its riders are;
- which orders are active;
- which rider has which order;
- what each rider is doing;
- which vehicle is available;
- which customer ordered what.

Yspace does not need to duplicate all of this.

The enterprise can send Yspace only what Yspace needs to solve the routing problem.

For example:

Origin
Destination
Stops
Vehicle profile

Departure time
Constraints

Yspace returns:

Recommended route
Distance
Estimated duration
ETA
Alternatives
Warnings
Route confidence

The company's own software then handles the rest.

21. DRIVER/RIDER EXPERIENCE

The driver or rider is **not the primary Yspace customer**.

This must be clear.

However, the rider may still interact with Yspace-powered intelligence through the enterprise's existing application.

For example:

ENTERPRISE APP

NEXT DELIVERY

Customer:
John

ETA:
3:42 PM

Recommended Route:
Route B

Yspace intelligence:
+8 min faster than alternative

The rider may never know that the route intelligence came from Yspace.

That is acceptable.

In fact, it is desirable.

Yspace becomes infrastructure.

22. OPTIONAL YSPACE DRIVER SDK

In the future, Yspace may provide an SDK or navigation component that enterprises can embed inside their own rider/driver application.

This should not be confused with Yspace managing the rider.

The enterprise still owns:

- the rider account;
- rider identity;
- employment relationship;
- vehicle;
- delivery assignment;
- operational system.

Yspace simply supplies navigation intelligence.

23. NAVIGATION OUTPUT

The routing engine can return:

- route geometry;
- turn-by-turn instructions;
- road segments;
- distance;
- duration;
- ETA;
- alternatives;
- warnings;
- confidence.

The enterprise decides how to present those outputs.

It may display them in:

- its existing driver app;
- its existing fleet platform;

- an embedded Yspace SDK;
- an approved Yspace navigation interface.

24. AUTONOMOUS ROUTE INTELLIGENCE

Now we define the actual core product.

Autonomous Route Intelligence is a system that takes transportation inputs and determines the most appropriate route according to defined objectives and constraints.

It should eventually operate across:

- motorcycles;
- cars;
- vans;
- trucks;
- other road vehicles supported by the platform.

25. BASIC ROUTE REQUEST

A request may contain:

Origin
Destination
Vehicle Type
Departure Time
Optimization Objective
Constraints

Example:

Origin:
Lagos Island

Destination:
Ikeja

Vehicle:
Motorcycle

Departure:

08:30

Objective:
Fastest reliable route

26. MULTI-STOP ROUTING

The enterprise may submit:

Depot
Stop A
Stop B
Stop C
Stop D
Stop E

The engine can calculate an efficient sequence.

This is a standard class of vehicle-routing problem. Current commercial routing systems already expose optimization capabilities for multi-location problems, including Mapbox's Optimization API and Google's Route Optimization API.

Therefore, Yspace should treat multi-stop optimization as a foundational capability rather than the entire competitive moat.

27. ROUTING OBJECTIVES

The enterprise should be able to specify the objective.

Examples:

Fastest

Minimize expected travel time.

Shortest

Minimize distance.

Reliable

Prefer routes with lower travel-time variability.

Balanced

Balance travel time and distance.

Avoid specific conditions

Avoid:

- tolls;
- highways;
- restricted roads;
- ferries;
- specific road types.

Google's routing infrastructure, for example, supports configurable objectives and route constraints such as time windows, vehicle capacity and route modifiers.

Yspace should expose similar concepts through a simpler logistics-oriented interface.

28. VEHICLE PROFILES

Vehicle information is necessary because different vehicles may need different routes.

A motorcycle route can differ from a truck route.

The system should therefore support:

Motorcycle profile

Car profile

Van profile

Truck profile

The profile should define routing characteristics rather than become a fleet-management record.

Potential parameters:

- travel mode;
- dimensions;
- weight;

- road restrictions;
- preferred roads;
- prohibited roads;
- operational assumptions.

29. TRAFFIC INTELLIGENCE

Traffic should be considered when calculating routes.

A route that is shortest geographically may not be fastest operationally.

The engine should compare:

```
Distance
+
Current Traffic
+
Historical Traffic
+
Expected Travel Time
```

Mapbox's current traffic-aware routing, for example, provides routing informed by current and historical traffic conditions.

Yspace can use third-party traffic/map infrastructure initially while progressively building proprietary intelligence.

30. ROUTE ALTERNATIVES

Yspace should not only produce one answer.

Where appropriate, it should generate alternatives.

Example:

Recommended

41 min

18.2 km

Alternative A

45 min

16.7 km

Alternative B

48 min

20.1 km

The system should explain why the first route was selected.

31. ROUTE EXPLANATION

This is important for trust.

The enterprise should see:

Recommended because it is expected to be 7 minutes faster under current traffic conditions.

Or:

Recommended because the shorter route has significantly higher congestion risk.

The user should not need to understand the optimization algorithm.

32. ROUTE SCORE

Yspace should eventually calculate a route score.

Example:

Route Reliability: 91/100

Potential inputs:

- historical travel-time variance;

- current traffic;
- road conditions;
- route stability;
- ETA confidence;
- known disruptions.

The score should only be displayed once the underlying model has sufficient validation.

33. ETA INTELLIGENCE

ETA should be a major product capability.

The system should distinguish between:

Estimated travel duration

and:

Estimated arrival time.

For example:

Travel time: 38 minutes

ETA: 9:18 AM

34. ETA CONFIDENCE

Eventually:

ETA: 9:18 AM

Confidence: High

or:

ETA: 9:18–9:26 AM

Confidence: Medium

This is more useful than pretending every prediction is exact.

35. ENTERPRISE ROUTE DASHBOARD

The enterprise dashboard should be designed around intelligence rather than fleet administration.

The main dashboard should show:

AUTONOMOUS ROUTE INTELLIGENCE

Routes analyzed
12,482

Average ETA accuracy
91%

Routes optimized
8,921

Estimated travel time improvement
12%

API requests
284,920

ROUTE INTELLIGENCE

Current analysis
Recent routes
Route performance
ETA performance
Alerts

All figures above are illustrative.

The production interface must use real data.

36. ROUTE WORKSPACE

This should be the flagship enterprise screen.

YSPACE

ENTERPRISE

ROUTE INPUT	MAP	INTELLIGENCE
Origin		Recommended
Destination		41 min
Stops		18.2 km
Vehicle		ETA 9:18 AM
Departure		
Objective		
ALTERNATIVE ROUTES		

37. ROUTE INPUT PANEL

The left panel should allow:

- origin;
- destination;
- multiple stops;
- vehicle profile;
- departure time;
- optimization objective;
- constraints.

The interface should support both:

Manual entry

and:

API-fed data.

38. MAP

The map is the central visualization.

It should display:

- origin;
- destination;
- stops;

- recommended route;
- alternatives;
- traffic;
- route warnings;
- selected route;
- route distance;
- route direction.

The map should remain visually dominant.

39. INTELLIGENCE PANEL

The right-hand panel should answer:

What does Yspace recommend?

Why?

How much better is it?

What risks exist?

What is the ETA?

Example:

Recommended Route

41 min

18.2 km

Why?

Current traffic on the alternative route is predicted to increase travel time by approximately 8 minutes.

40. ROUTE COMPARISON

When alternatives exist:

RECOMMENDED
41 min
18.2 km
Reliability 91

ALTERNATIVE
49 min
16.5 km
Reliability 72

This communicates a key concept:

Shortest does not always mean best.

41. ROUTE HISTORY

Enterprise users should be able to see previously calculated routes.

Each record:

- date;
- origin;
- destination;
- vehicle profile;
- duration;
- distance;
- recommended route;
- alternatives;
- result.

This creates the foundation for future route learning.

42. ROUTING PROFILES

Businesses may have different routing requirements.

For example:

Motorcycle — Urban

Van — Urban

Truck — Regional

Each profile can contain:

- vehicle type;
- routing preferences;
- constraints;
- optimization objective.

This allows the enterprise to standardize its routing behavior.

43. ENTERPRISE SETTINGS

Settings should include:

Organization

Users

Roles

Routing Profiles

API

Webhooks

Billing

Security

Data

Integrations

44. API AS THE CORE ENTERPRISE PRODUCT

The enterprise should be able to use Yspace without manually visiting the dashboard for every route.

The API should support programmatic routing.

Conceptually:

POST /route

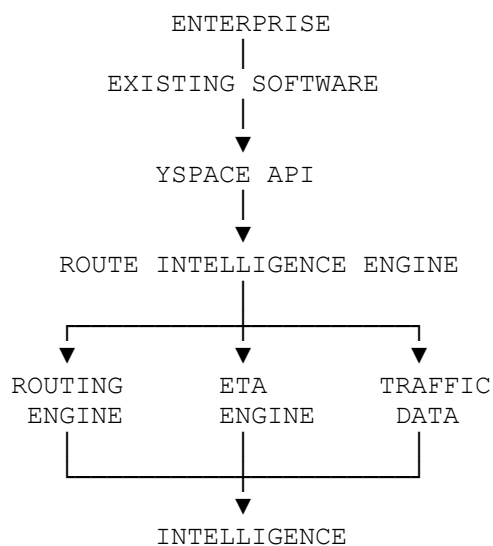
Request:

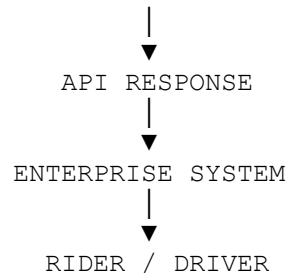
origin
destination
stops
vehicle_profile
departure_time
constraints
objective

Response:

route_id
recommended_route
distance
duration
eta
geometry
alternatives
warnings
confidence

45. API INTEGRATION FLOW





46. SDK

Yspace should consider SDKs for:

- Android;
- iOS;
- web.

The SDK should allow enterprises to embed:

- route visualization;
- navigation;
- ETA;
- route updates;
- route instructions.

Again, the enterprise retains ownership of the driver/rider application.

47. WEBHOOKS

The enterprise can subscribe to events such as:

- route calculated;
- route recalculated;
- ETA updated;
- route warning;
- route risk;
- API usage event.

This allows the enterprise system to react automatically.

48. DATA MODEL

The core entities should include:

```
User
Organization
Subscription
RoutingProfile
RouteRequest
Route
RouteAlternative
Stop
ETA
RouteEvent
APIKey
Webhook
Integration
UsageRecord
```

There should not be a requirement to create:

```
Yspace Driver
Yspace Rider
Yspace Vehicle
```

for enterprise routing merely because the enterprise operates those assets.

Those remain external entities unless an integration requires limited identifiers to calculate a route.

49. EXTERNAL VEHICLE REPRESENTATION

The API may receive:

```
vehicle_type: motorcycle
```

or:

```
vehicle_profile: enterprise_motorcycle_urban
```

But this does not mean Yspace owns or manages the motorcycle.

It is simply a routing parameter.

50. DRIVER IDENTITY

Similarly, an enterprise might send:

```
driver_id: RIDER_238
```

only if necessary for the enterprise integration.

Yspace should not become the system of record for that driver.

The identifier belongs to the enterprise.

51. SECURITY MODEL

Enterprise data can be sensitive.

The platform should support:

- organization isolation;
- role-based access;
- API authentication;
- API key rotation;
- encrypted communication;
- audit logs;
- access logs;
- webhook signing;
- usage controls.

52. API AUTHENTICATION

Enterprise API access should use secure credentials.

Potential structure:

```
Organization
├── Production API Key
└── Test API Key
```

└─ Webhook Secret

Keys should be:

- revocable;
- rotatable;
- scoped;
- auditable.

53. DATA OWNERSHIP

Enterprise customers should clearly understand what happens to their data.

The product should distinguish:

Customer operational data

Data submitted by the enterprise.

Yspace-generated intelligence

Route calculations, predictions and derived outputs.

Platform data

System-level information required to operate Yspace.

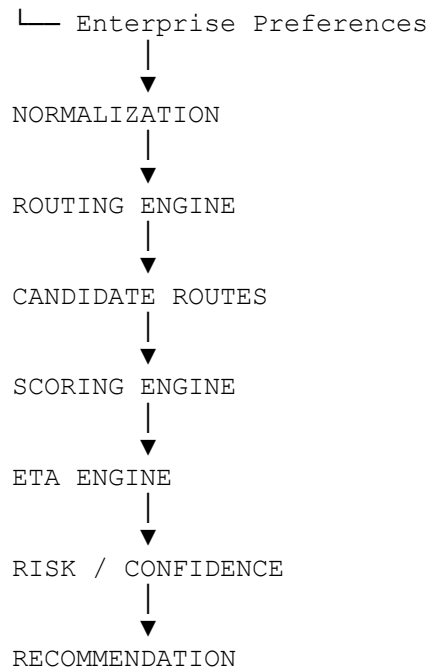
The final legal/data policy must be defined separately with legal counsel.

54. ROUTE INTELLIGENCE ENGINE

At a high level:

INPUTS

├─ Location
├─ Road Network
├─ Traffic
├─ Vehicle Profile
├─ Departure Time
├─ Constraints
├─ Historical Data



55. EXTERNAL MAP DATA

Yspace should not necessarily attempt to build the entire global road network from scratch.

Third-party providers can provide foundational infrastructure.

For example, Mapbox currently provides:

- directions;
- traffic-aware directions;
- matrix calculations;
- map matching;
- optimization;
- navigation SDKs.

Google similarly provides route optimization infrastructure supporting transportation objectives and constraints.

The Yspace engineering strategy should therefore focus proprietary effort where it creates differentiated value.

56. WHERE YSPACE SHOULD BUILD PROPRIETARY INTELLIGENCE

Potential proprietary layers include:

Local logistics intelligence

Understanding how transportation actually behaves in Yspace's target markets.

ETA intelligence

Improving predicted travel time using Yspace's own historical observations.

Route reliability

Understanding which routes are consistently reliable.

Enterprise-specific routing

Learning how a specific business wants its routes optimized.

Logistics risk

Predicting potential route failures or delays.

Route explanations

Turning complex optimization into understandable recommendations.

57. ENTERPRISE-SPECIFIC LEARNING

Suppose Company A consistently prioritizes:

reliability over absolute shortest distance.

Company B may prioritize:

minimum travel time.

Company C may prioritize:

fuel efficiency.

Yspace should eventually allow these preferences to be represented.

The system can therefore become personalized to each enterprise.

58. ROUTE PERFORMANCE LEARNING

After an enterprise uses a recommended route, Yspace can eventually compare:

Predicted:
42 minutes

Actual:
47 minutes

Over thousands of observations, Yspace can learn where predictions are systematically wrong.

This creates a feedback loop.

```
PREDICTION
  ↓
ACTUAL RESULT
  ↓
ERROR
  ↓
MODEL LEARNING
  ↓
BETTER PREDICTION
```

This is where the product begins to move beyond ordinary map routing.

59. PREDICTIVE ETA

A future intelligence layer can calculate:

ETA:
4:38 PM

Expected range:
4:35-4:43 PM

Confidence:
87%

The confidence score should be calibrated against historical model performance.

60. LOGISTICS RISK INTELLIGENCE

The system can eventually identify:

High congestion risk

ETA uncertainty

Route instability

Unusual traffic

Delivery-window risk

Abnormal travel time

Route disruption

The risk engine should not simply produce alerts.

It should produce actionable intelligence.

Example:

High delay risk detected.

Alternative route is expected to reduce delay probability.

61. PREDICTIVE TAGGING

Routes can automatically receive tags:

HIGH CONGESTION
LOW ETA CONFIDENCE
RELIABLE ROUTE
HIGH DELAY RISK
UNUSUAL TRAVEL TIME
ALTERNATIVE RECOMMENDED

The tags become a visual language throughout the dashboard.

62. ROUTE RELIABILITY

A route may be geographically attractive but operationally unreliable.

Yspace should eventually distinguish:

Fastest route

from:

Most reliable route

from:

Best overall route

This distinction could become one of the core product concepts.

63. "BEST ROUTE" SCORE

Potential scoring:

Travel Time	40%
Reliability	25%
Traffic	15%
Distance	10%
Constraints	10%

These values are examples, not final weights.

The system should eventually allow enterprise-specific weighting.

64. WHY THE PRODUCT IS "AUTONOMOUS"

The word autonomous should be used carefully.

Autonomous does not necessarily mean:

Yspace physically controls the vehicle.

For enterprise routing, autonomy means the intelligence can automatically:

- evaluate options;
- calculate routes;
- compare alternatives;
- predict travel time;
- identify risks;
- recommend decisions;
- update recommendations.

The enterprise remains in control of its own operations.

65. AUTOMATIC ROUTE RECALCULATION

An enterprise integration could send:

"Vehicle is currently here."

Yspace can recalculate:

"The original route is no longer optimal."

and return:

"Use Route B."

The enterprise decides whether to apply that recommendation automatically.

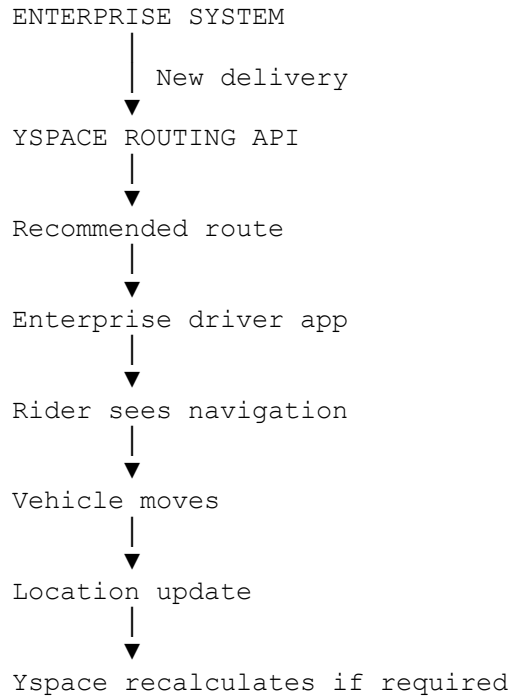
This is an important distinction.

Yspace provides the intelligence.

The enterprise controls the execution policy.

66. DRIVER NAVIGATION FLOW

A possible integrated experience:



67. DO NOT BUILD A RIDER MANAGEMENT SYSTEM

This requirement should be explicit in the PRD.

Yspace should **not** attempt to replicate:

- employee management;
- rider attendance;
- payroll;
- rider recruitment;
- driver scheduling;
- rider performance management;
- fleet maintenance;
- vehicle ownership management.

Those are outside the primary scope of Autonomous Route Intelligence.

68. DO NOT BUILD A GENERIC FLEET MANAGEMENT SYSTEM

The enterprise may already have one.

Yspace should integrate with it.

This makes Yspace more attractive because the customer does not need to replace its existing infrastructure.

69. ENTERPRISE INTEGRATION OPTIONS

Three levels should eventually exist.

Level 1 — Dashboard

Enterprise manually enters routes.

Best for:

- testing;
- small businesses;
- sales demonstrations.

Level 2 — API

Enterprise's system automatically requests routes.

Best for:

- medium and large businesses;
- production operations.

Level 3 — SDK

Enterprise embeds Yspace navigation directly into its application.

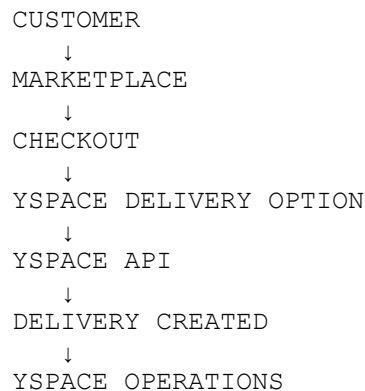
Best for:

- large logistics platforms;
- marketplaces;
- delivery networks.

70. MARKETPLACE INTEGRATION

A marketplace could integrate Yspace's physical delivery service.

Example:



This is different from Autonomous Route Intelligence.

Here, Yspace is performing the delivery.

The marketplace is outsourcing transportation to Yspace.

71. MARKETPLACE ROUTE INTELLIGENCE

A marketplace could potentially use both products.

For example:

Product A

Yspace physically delivers selected orders.

Product B

The marketplace uses Yspace routing intelligence for its own ground fleet.

The same business can therefore use both Yspace products.

72. DRONE DELIVERY BOOKING

The normal consumer should see:

Send a Package

not:

Autonomous Logistics Platform.

The product language must match the user's goal.

73. PACKAGE INFORMATION

The booking system should collect only the information needed to determine:

- whether Yspace can transport the package;
- price;
- serviceability;
- operational requirements.

Potential information:

- weight;
- dimensions;
- package type;
- declared value;
- special handling.

74. DELIVERY PRICE

The system should calculate pricing based on actual Yspace commercial rules.

Potential factors include:

- distance;
- package characteristics;
- service type;
- operational cost;
- location;
- demand;
- other business rules.

Pricing should not be hardcoded into the UI.

The backend should return the price.

75. DELIVERY ETA

The customer should receive:

Estimated arrival: 4:35 PM

The underlying estimate may use:

- route;
- drone capability;
- mission preparation;
- pickup time;
- operational conditions;
- weather;
- current mission state.

The customer sees only the appropriate customer-safe result.

76. TRACKING

Tracking should show:

Current status

Estimated arrival

Location

Progress

Destination

The map should be simple.

The customer does not need Mission Control's operational complexity.

77. DELIVERY STATUS MODEL

Recommended:

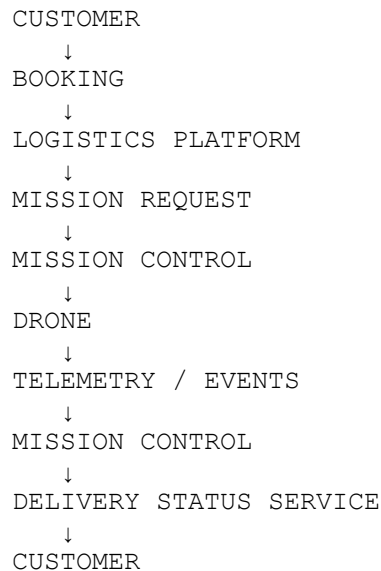
CREATED
↓
CONFIRMED
↓
PREPARING
↓
PICKUP
↓
READY
↓
IN TRANSIT
↓
APPROACHING
↓
DELIVERED

Exception states:

DELAYED
CANCELLED
FAILED
REQUIRES ACTION

78. MISSION CONTROL CONNECTION

The delivery platform should receive controlled information from Mission Control.



The customer should not directly connect to the drone.

79. CUSTOMER NOTIFICATIONS

Notifications may include:

Booking confirmed

Package received

Delivery started

Delivery approaching

Delivered

Delay

Action required

Notifications should be available through:

- push;
- SMS;

- email;
- WhatsApp where supported and commercially appropriate.

80. ENTERPRISE NOTIFICATIONS

Enterprise users may receive:

- API alerts;
- usage alerts;
- system status;
- route risk events;
- ETA changes.

The system should avoid flooding enterprise users with unnecessary notifications.

81. MOBILE APPLICATION

The mobile application should primarily serve consumer delivery use cases.

The enterprise experience can be responsive and accessible through mobile where appropriate, but the primary enterprise workspace should be designed for desktop/web because route analysis benefits from larger displays.

The enterprise API should remain the primary method for production integrations.

82. WEB APPLICATION

The web application should support both:

Customer delivery

and:

Enterprise intelligence.

However, the two experiences should be clearly separated.

83. MOBILE INFORMATION ARCHITECTURE

HOME

- Send Package
- Active Delivery
- Track
- History
- Notifications
- Account
- Support

Enterprise users with appropriate permissions may additionally access:

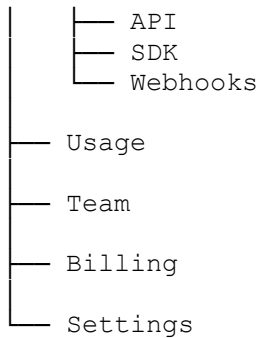
ENTERPRISE

- Overview
- Route Intelligence
- History
- Integrations
- Settings

84. ENTERPRISE WEB INFORMATION ARCHITECTURE

ENTERPRISE

- Overview
- Route Intelligence
 - New Route
 - Multi-Stop
 - Alternatives
 - Route Results
- Routes
 - Recent
 - Saved
 - History
- Routing Profiles
- Analytics
- Integrations



85. DASHBOARD DESIGN REQUIREMENTS

The dashboard should not be overloaded.

The first screen should answer:

What is happening?

What has happened?

What needs attention?

What can I do now?

Primary CTA:

Analyze a Route

Secondary:

View API

86. ROUTE ANALYSIS SCREEN

The route analysis screen should be the core interaction.

Inputs:

- origin;

- destination;
- stops;
- vehicle profile;
- departure time;
- optimization preference.

Output:

- recommended route;
- ETA;
- distance;
- alternatives;
- confidence;
- warnings;
- explanation.

87. ROUTE ANALYSIS STATES

The UI should support:

Empty

"No route analyzed yet."

Loading

"Analyzing route..."

Result

"Recommended route."

Warning

"Route requires attention."

Error

"Unable to calculate route."

No route

"No suitable route found."

88. LOADING EXPERIENCE

Routing may require multiple services.

The UI should not simply display a generic spinner.

It can show:

Analyzing road network...

Checking current conditions...

Comparing route alternatives...

Calculating ETA...

Then:

Recommendation ready.

The exact animation should remain subtle.

89. MAP INTERACTION

Users should be able to:

- zoom;
- pan;
- select routes;
- select stops;
- inspect alternatives;
- view route information.

Clicking a route should update the intelligence panel.

90. ROUTE DETAILS

When a route is selected:

ROUTE DETAILS

Distance
18.2 km

Estimated time
41 min

ETA
9:18 AM

Reliability
91

Traffic
Moderate

Recommendation
Preferred

91. ENTERPRISE ANALYTICS

Analytics should eventually include:

Route volume

Average travel time

ETA performance

Route improvement

Route reliability

API usage

Cost/usage

Analytics should be based on actual available data.

The UI must never imply savings that have not been measured.

92. SAVED ROUTES

Enterprise users should be able to save common routes.

Example:

Lagos Island → Ikeja Warehouse

This allows quick re-analysis.

93. SAVED ROUTING PROFILES

Examples:

Motorcycle Urban

Van Urban

Truck Regional

The enterprise can reuse these profiles.

94. API TEST CONSOLE

The enterprise workspace should eventually provide an API testing environment.

The developer can:

1. choose endpoint;
2. enter request;
3. send request;
4. inspect response.

Example:

POST /route

REQUEST

```
{
  origin,
  destination,
  vehicle_profile,
  departure_time
}
```

RESPONSE

```
{
  route,
  duration,
  distance,
  eta
}
```

95. API USAGE

Dashboard:

API USAGE

Requests today
18,421

This month
482,190

Plan
Enterprise

Remaining
...

Actual pricing/limits depend on the commercial model.

96. WEBHOOK CONFIGURATION

Enterprise users can configure:

Webhook URL
Secret
Events
Status

Example events:

route.created
route.updated
eta.updated
risk.detected

97. ENTERPRISE BILLING

Billing should be separated from routing.

Possible commercial models:

Subscription

Monthly platform fee.

Usage

Charge based on routing requests.

Enterprise contract

Custom pricing.

The final model can support combinations.

98. PRICING UX

Public enterprise page:

Autonomous Route Intelligence

Built for logistics companies that want better routing intelligence without replacing their existing fleet systems.

CTA:

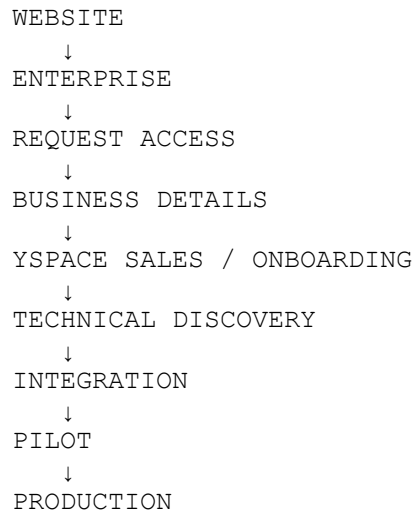
Talk to Yspace

or:

Request Access

This is more appropriate for early enterprise sales than forcing every company into a self-serve pricing table.

99. ENTERPRISE SALES FLOW



100. PILOT MODE

A logistics company should be able to test Yspace before full deployment.

Pilot capabilities:

- limited API requests;
- test routing;
- route comparison;
- sandbox environment;
- limited users;
- sample data.

This allows Yspace to demonstrate value without forcing immediate full integration.

101. ROUTE BENCHMARKING

A powerful enterprise feature is comparing:

Existing route

versus:

Yspace recommendation.

Example:

CURRENT ROUTE
52 min

YSPACE ROUTE
43 min

Difference
9 min

But the system should only calculate a real improvement when it has comparable conditions and a defensible baseline.

This is critical for credibility.

102. ENTERPRISE VALUE PROPOSITION

The enterprise should be able to understand:

"Yspace does not replace our logistics system."

Instead:

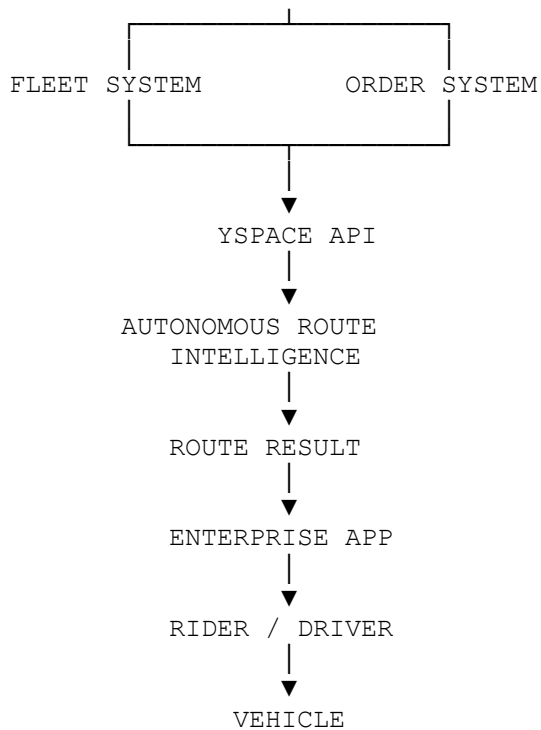
"Yspace makes our existing logistics system smarter."

This should be central to the product positioning.

103. THE DRIVER EXPERIENCE — FINAL ARCHITECTURE

The correct architecture is:

ENTERPRISE
|



Yspace does not need to own the bottom half.

That is the point.

104. WHY THIS IS STRATEGICALLY IMPORTANT

If Yspace tried to become the fleet-management system for every customer, every enterprise would have to replace existing infrastructure.

That creates enormous adoption friction.

Instead, Yspace can become an intelligence layer.

This means an enterprise can potentially integrate Yspace without replacing:

- its fleet-management platform;
- its order-management system;
- its driver application;
- its dispatch platform.

That dramatically changes the integration proposition.

105. YSPACE AS LOGISTICS INTELLIGENCE INFRASTRUCTURE

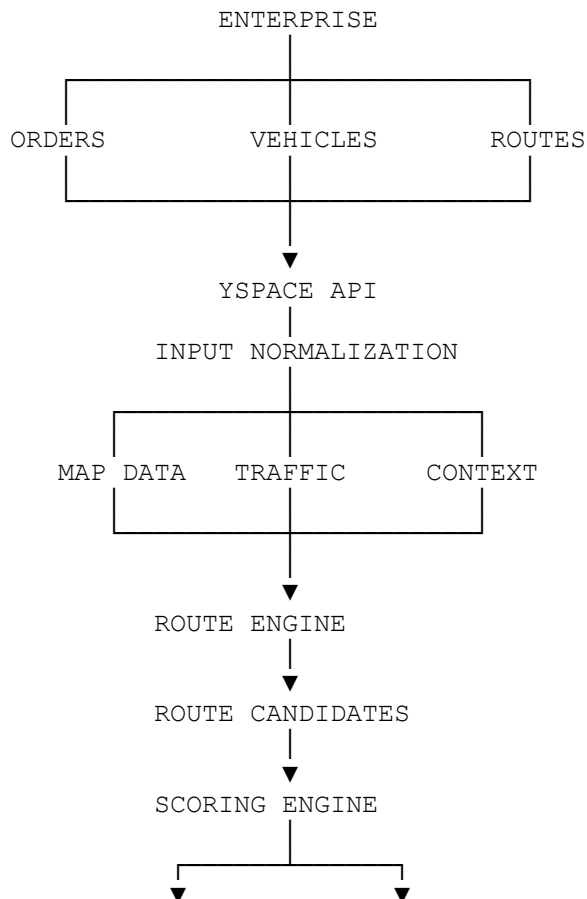
The long-term positioning becomes:

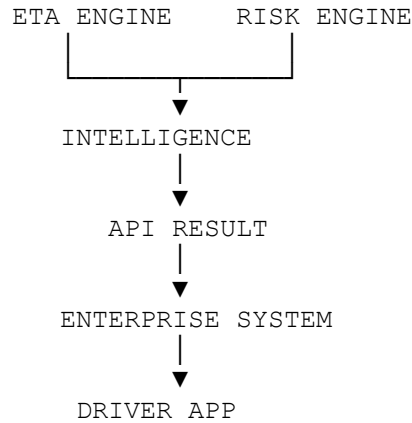
Yspace provides the intelligence layer behind transportation decisions.

That is much larger than:

"Yspace is another navigation app."

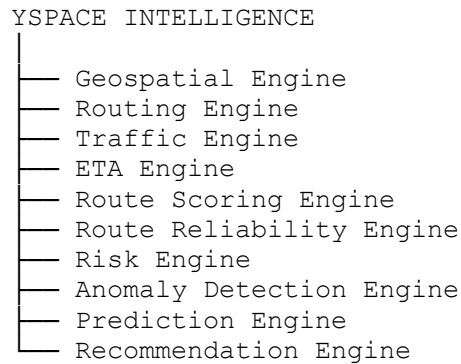
106. DATA FLOW — COMPLETE ENTERPRISE SYSTEM





107. AI ENGINE ARCHITECTURE

The intelligence architecture should eventually consist of separate engines rather than one giant "AI."



Each engine should have a clear responsibility.

108. GEOSPATIAL ENGINE

Responsibilities:

- coordinates;
- road network;
- map matching;
- geographic boundaries;
- location normalization;
- route geometry.

109. ROUTING ENGINE

Responsibilities:

- calculate routes;
- generate alternatives;
- support vehicle profiles;
- support constraints;
- support multi-stop routing.

110. TRAFFIC ENGINE

Responsibilities:

- current traffic;
- historical traffic;
- traffic patterns;
- traffic-aware duration.

Where third-party providers are used, Yspace should treat those inputs as infrastructure rather than claiming the underlying data as proprietary.

111. ETA ENGINE

Responsibilities:

- calculate estimated travel time;
- calculate arrival time;
- update predictions;
- measure prediction error.

112. ROUTE SCORING ENGINE

Responsibilities:

- compare candidates;
- apply enterprise preferences;
- score reliability;
- rank alternatives.

113. RISK ENGINE

Responsibilities:

- identify route risks;
- estimate probability of delays;
- identify unusual conditions.

This should be introduced only when sufficient data exists to make predictions responsibly.

114. ANOMALY ENGINE

Responsibilities:

- detect unusual travel times;
- identify deviations from expected behavior;
- flag abnormal route performance.

115. RECOMMENDATION ENGINE

This is the user-facing intelligence layer.

It converts:

Complex calculations
+
Data
+
Predictions

into:

Recommendation

For example:

Route B recommended.

Expected to save approximately 8 minutes.

Current traffic is the primary reason.

116. VERSIONING — CORRECT INTERPRETATION

This PRD does **not** divide the product into "Version 1" and "Version 2."

The product is one platform.

Instead, features should be classified internally as:

Foundation

Required for the core product.

Advanced

Requires more data or infrastructure.

Future Intelligence

Requires validated models and larger datasets.

This avoids artificially splitting the product into separate products.

117. FOUNDATION CAPABILITIES

The core platform should include:

- account system;
- customer booking;
- delivery tracking;
- enterprise organization;

- enterprise workspace;
- route calculation;
- route alternatives;
- ETA;
- routing profiles;
- API;
- map;
- integration infrastructure.

118. ADVANCED CAPABILITIES

Advanced functionality includes:

- multi-stop optimization;
- route comparison;
- route scoring;
- enterprise-specific preferences;
- historical route analysis;
- SDK;
- dynamic recalculation.

119. FUTURE INTELLIGENCE

Future intelligence can include:

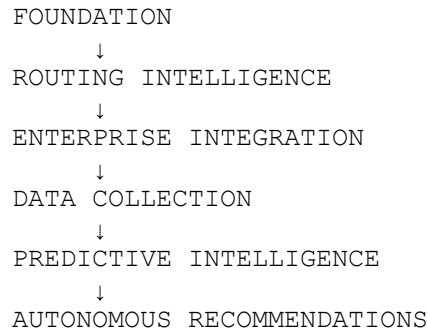
- predictive ETA;
- logistics risk engine;
- predictive route tagging;
- anomaly detection;
- route reliability prediction;
- learned local routing;
- proactive recommendations.

These are not "Version 2."

They are **future intelligence layers of the same product.**

120. PRODUCT ROADMAP MODEL

The roadmap should therefore look like:



Not:

V1 → V2

121. UI DESIGN PRIORITIES

The designer should prioritize these screens first:

Priority 1

Consumer:

- Home;
- Send Package;
- Booking;
- Estimate;
- Checkout;
- Tracking.

Priority 2

Enterprise:

- Enterprise landing;
- Enterprise onboarding;
- Enterprise dashboard;
- Route Intelligence workspace;
- Route results;

- Route comparison.

Priority 3

Enterprise infrastructure:

- Routing Profiles;
- API;
- Webhooks;
- Integrations;
- Analytics;
- Usage;
- Team;
- Billing.

122. DESIGN SYSTEM

The Yspace design system should include:

Typography

Clear hierarchy.

Color

Functional use of color.

For example:

- normal;
- success;
- warning;
- critical;
- informational.

Cards

For metrics and route intelligence.

Map overlays

For route status and traffic.

Buttons

Clear action hierarchy.

Inputs

Fast location entry.

Status indicators

Consistent across delivery and enterprise experiences.

123. MAP DESIGN LANGUAGE

The map is not decoration.

It is a functional interface.

The designer should define:

- route colors;
- route selection;
- alternatives;
- stop markers;
- traffic;
- warnings;
- selected segments;
- map labels.

The visual system must remain understandable when multiple routes overlap.

124. MOBILE MAP

Mobile maps should prioritize:

- current route;
- next action;
- ETA;
- destination.

The map should not become cluttered with enterprise analytics.

125. DESKTOP MAP

The enterprise map can be much richer.

It may display:

- route alternatives;
- route scores;
- traffic;
- multiple stops;
- route segments;
- route comparison.

126. EMPTY STATES

Enterprise empty state:

No routes analyzed yet.

Create your first route to see Yspace's routing intelligence in action.

CTA:

Analyze a Route

Consumer empty state:

No deliveries yet.

CTA:

Send a Package

127. ERROR STATES

Errors should explain what happened.

Bad:

Error 500.

Better:

We couldn't calculate this route right now.

Try again or check the locations.

128. API ERROR DESIGN

API errors should be machine-readable and human-readable.

Example:

```
ROUTE_UNAVAILABLE
```

```
message:
```

```
No suitable route could be calculated between the requested locations.
```

129. OBSERVABILITY

Engineering should track:

- API latency;
- routing failures;
- map provider failures;
- ETA service failures;
- integration failures;
- webhook failures.

The product should have internal monitoring separate from the customer interface.

130. PERFORMANCE REQUIREMENTS

The route experience should feel fast.

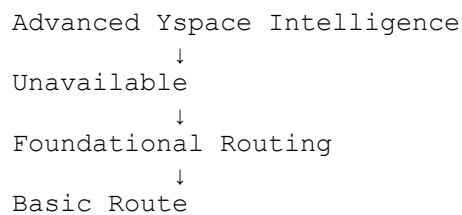
The system should:

- cache where appropriate;
- avoid unnecessary map requests;
- parallelize independent data requests;
- use asynchronous processing for complex optimization;
- gracefully degrade when non-critical intelligence is unavailable.

131. FALLBACK ROUTING

If the advanced intelligence layer is unavailable, the platform should ideally fall back to foundational routing infrastructure where technically and contractually possible.

For example:

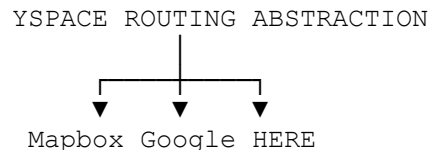


This prevents the entire product from becoming unusable because one intelligence service fails.

132. PROVIDER-AGNOSTIC ARCHITECTURE

The backend should avoid tightly coupling the product to one mapping provider.

Conceptually:



The exact providers depend on:

- coverage;

- pricing;
- technical performance;
- licensing;
- African market quality;
- traffic availability;
- contractual terms.

This architecture also protects Yspace from provider lock-in.

133. LOCALIZATION

Yspace should prioritize markets where local routing intelligence can become a competitive advantage.

The platform should support:

- local addresses;
- landmarks;
- local road naming;
- local transportation behavior;
- local traffic patterns.

This becomes particularly important in markets where conventional address systems may not fully represent how deliveries actually happen.

134. ADDRESS INTELLIGENCE

A logistics platform should eventually understand:

"Near this landmark."

"Behind this building."

"At this gate."

"Use this entrance."

These contextual signals can eventually improve delivery and routing accuracy.

135. ROUTE LEARNING

Every completed route can potentially produce learning data.

Conceptually:

```
REQUEST
↓
PREDICTION
↓
ROUTE
↓
ACTUAL MOVEMENT
↓
ACTUAL TIME
↓
COMPARISON
↓
MODEL IMPROVEMENT
```

This must be implemented with appropriate privacy, consent, contractual and data-governance controls.

136. ENTERPRISE FEEDBACK

The enterprise should eventually be able to indicate:

Route recommendation was useful.

or:

Route recommendation was not useful.

This feedback can help improve the system.

137. HUMAN OVERRIDE

The system should never assume the algorithm is always correct.

Enterprise users should be able to override a recommendation.

For example:

Use Alternative Route

The platform should record the override where appropriate.

Over time, repeated overrides can reveal missing constraints.

138. CONSTRAINT LEARNING

If an enterprise repeatedly avoids a certain road, Yspace can eventually ask:

"Would you like to permanently avoid this road?"

This turns manual behavior into configuration.

139. ROUTE EXPLANABILITY

The system should maintain an explanation object internally.

For example:

Recommendation:
Route B

Primary reason:
Lower predicted travel time

Secondary reason:
Lower traffic volatility

Constraint:
Truck restriction respected

The UI converts this into simple language.

140. ENTERPRISE TRUST

Enterprise logistics decisions can affect thousands of deliveries.

Therefore, the system must prioritize:

- accuracy;
- transparency;
- reliability;
- explainability;
- observability.

A black-box recommendation without explanation is less useful to a serious logistics company.

141. ENTERPRISE API CONTRACT

The API specification should eventually define:

Inputs

- coordinates;
- addresses;
- stops;
- vehicle profile;
- departure;
- constraints;
- objective.

Outputs

- route;
- alternatives;
- duration;
- distance;
- ETA;
- confidence;
- warnings;
- explanation.

142. ROUTE REQUEST EXAMPLE

Origin:
6.45, 3.39

Destination:
6.60, 3.35

Vehicle:
MOTORCYCLE_URBAN

Departure:
2026-08-31 08:30

Objective:
RELIABLE_FASTEST

The backend should return the best available recommendation.

143. ROUTE RESPONSE EXAMPLE

Recommended Route

Distance:
18.2 km

Duration:
41 min

ETA:
09:11

Confidence:
High

Alternative:
46 min

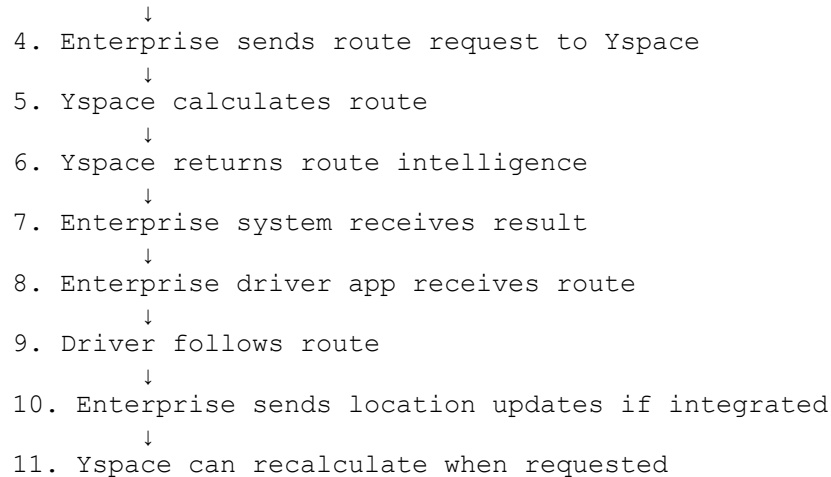
Reason:
Lower predicted congestion.

These are illustrative values only.

144. ENTERPRISE API FLOW WITH DRIVER

The complete flow should be:

1. Customer places order
↓
2. Enterprise system receives order
↓
3. Enterprise determines delivery vehicle



Yspace does not need to own steps 1, 2, 3, 7 or 8.

145. WHAT YSPACE DOES NOT NEED TO KNOW

Depending on the integration, Yspace may not need:

- driver's name;
- driver's salary;
- driver's employment details;
- driver's personal information;
- vehicle maintenance history;
- payroll;
- HR data.

Only the information necessary to provide routing intelligence should be shared.

146. DRONE DELIVERY VS ENTERPRISE ROUTING

The UI must clearly distinguish:

"Send with Yspace"

Yspace performs delivery.

"Use Yspace Intelligence"

Your company uses Yspace's intelligence for its own logistics.

This wording is more important than simply labeling both:

Logistics.

147. CUSTOMER ACCOUNT

Customer account:

Personal profile
Delivery history
Payment
Addresses
Notifications

148. BUSINESS ACCOUNT

Business account:

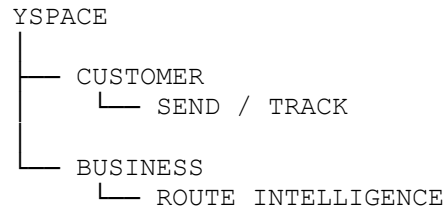
Organization
Team
Subscription
API
Routing Profiles
Usage
Analytics
Integrations

This is a completely different information architecture.

149. SAME PLATFORM, DIFFERENT EXPERIENCES

They can exist within one Yspace ecosystem.

But they should not be forced into one confusing workflow.



150. WHY THE TWO EXPERIENCES CAN LIVE TOGETHER

There is strategic value in keeping them under the same platform.

The company becomes:

Yspace — Autonomous Logistics & Intelligence

rather than:

A drone company with a random routing SaaS product.

The routing intelligence is connected to the company's deeper logistics technology.

151. BUSINESS LANDING PAGE

Headline:

Make every route smarter.

Subheadline:

Yspace Autonomous Route Intelligence helps logistics companies calculate faster, more reliable routes and integrate intelligent routing directly into their existing logistics systems.

CTA:

Talk to Yspace

Secondary:

Explore the Technology

152. ENTERPRISE DEMONSTRATION

A sales demo should show:

```
Existing enterprise route
↓
Yspace analysis
↓
Alternative routes
↓
ETA comparison
↓
Recommendation
↓
API integration
```

This is likely much more convincing to enterprise customers than simply showing a map.

153. INVESTOR-LEVEL PRODUCT STORY

The product story can be summarized as:

Yspace is building an autonomous logistics company with its own operating system.

The company has:

Physical layer

Drone transportation.

Intelligence layer

Autonomous Route Intelligence.

Operating layer

ySpaceOS.

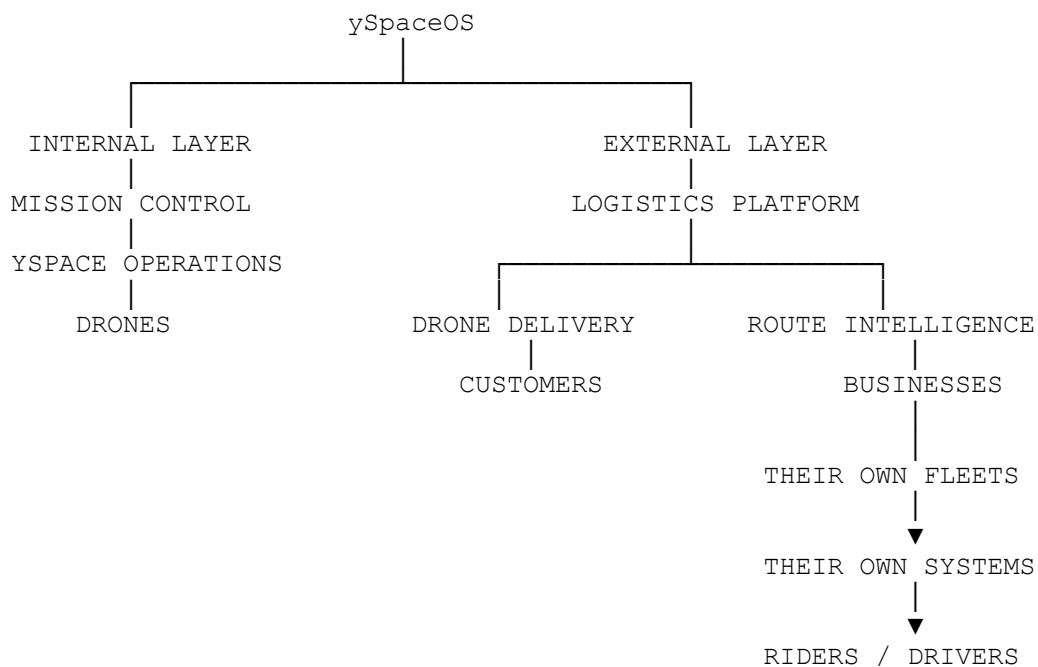
Control layer

Mission Control.

Commercial layer

Yspace Logistics Platform.

154. COMPLETE YSPACE ARCHITECTURE



155. THE CENTRAL PRODUCT IDEA

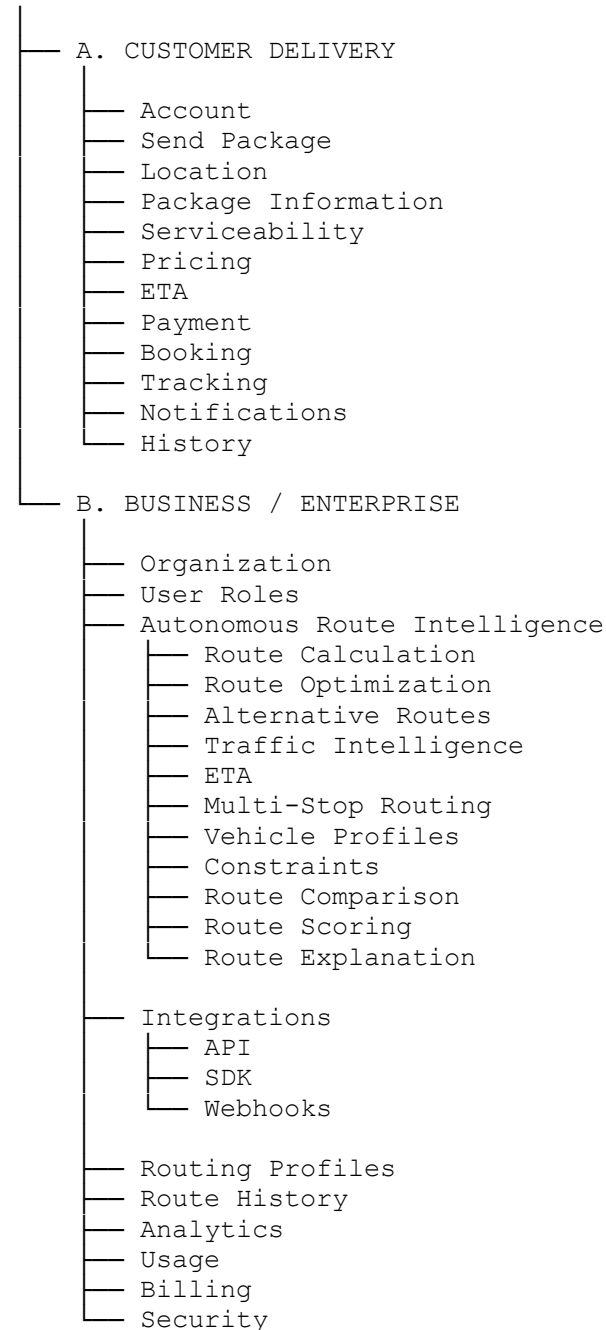
The most important sentence in this entire PRD is:

Yspace operates its own transportation infrastructure, while giving other businesses the intelligence to operate theirs better.

That is the cleanest definition of the platform.

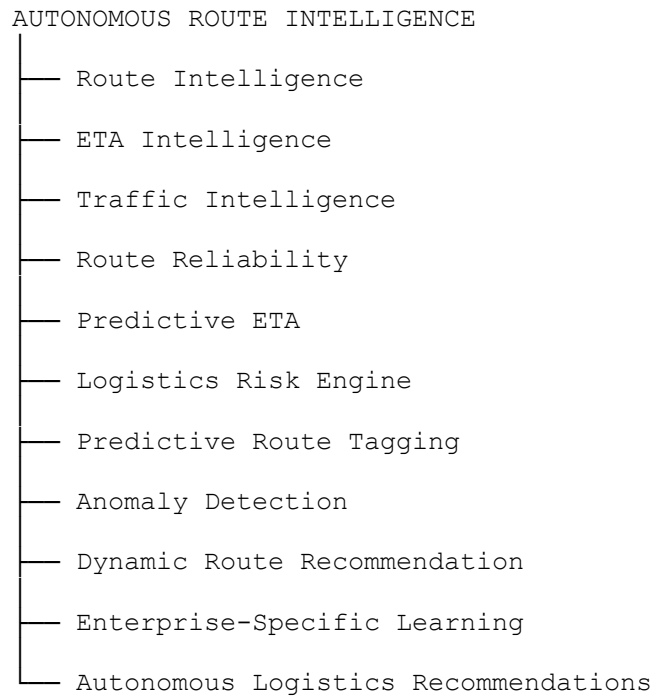
156. FINAL FEATURE HIERARCHY

YSPACE LOGISTICS PLATFORM



157. FUTURE INTELLIGENCE HIERARCHY

These capabilities should remain part of the **same Autonomous Route Intelligence product**:

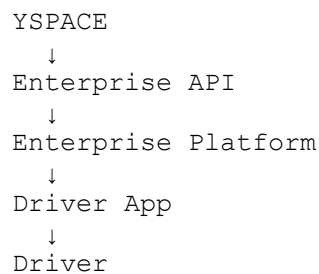


They are not separate products.

158. THE ROLE OF THE DRIVER

The driver/rider is an endpoint in the enterprise's system.

For example:



The driver does not need to have a Yspace account.

The driver does not need to subscribe to Yspace.

The driver does not need to open the Yspace dashboard.

The enterprise owns that relationship.

159. THE ROLE OF THE ENTERPRISE

The enterprise is the actual customer of Autonomous Route Intelligence.

It:

- subscribes;
- integrates;
- sends routing requests;
- receives recommendations;
- configures routing preferences;
- manages API access;
- reviews analytics.

160. THE ROLE OF YSPACE

Yspace:

- provides routing intelligence;
- calculates routes;
- provides ETA;
- processes geospatial information;
- provides APIs;
- provides SDKs;
- improves intelligence;
- maintains infrastructure.

161. THE ROLE OF ySpaceOS

ySpaceOS provides the underlying intelligence and operating infrastructure connecting Yspace's autonomous transportation systems.

It can power:

- Mission Control;

- route intelligence;
- geospatial intelligence;
- telemetry infrastructure;
- AI services;
- operational data;
- future autonomous systems.

162. THE ROLE OF MISSION CONTROL

Mission Control is Yspace's internal control tower.

It should not be confused with the enterprise platform.

Mission Control controls Yspace's own operations.

The Logistics Platform sells those capabilities externally.

163. THE LONG-TERM VISION

The long-term opportunity is not simply:

"Yspace delivers packages with drones."

It is:

Yspace builds the operating and intelligence infrastructure for autonomous logistics.

The drone operation becomes the physical demonstration of that capability.

The enterprise routing platform becomes the software commercialization of that intelligence.

164. PRODUCT FLYWHEEL

The long-term system can create:

YSPACE OPERATIONS

↓

REAL-WORLD LOGISTICS DATA

↓
BETTER MODELS
↓
BETTER ROUTING
↓
BETTER ETA
↓
BETTER DELIVERY
↓
MORE CUSTOMERS
↓
MORE DATA
↓
BETTER INTELLIGENCE

And separately:

ROUTE INTELLIGENCE
↓
ENTERPRISE CUSTOMERS
↓
MORE ROUTING DATA
↓
BETTER MODELS
↓
BETTER ROUTE INTELLIGENCE
↓
MORE ENTERPRISE CUSTOMERS

165. FINAL PRODUCT DEFINITION

The Yspace Logistics Platform is therefore a single external platform with two clearly differentiated commercial experiences.

CUSTOMER SIDE

Yspace Drone Delivery

A customer or business can ask Yspace to physically transport a package.

Yspace performs the delivery.

BUSINESS SIDE

Yspace Autonomous Route Intelligence

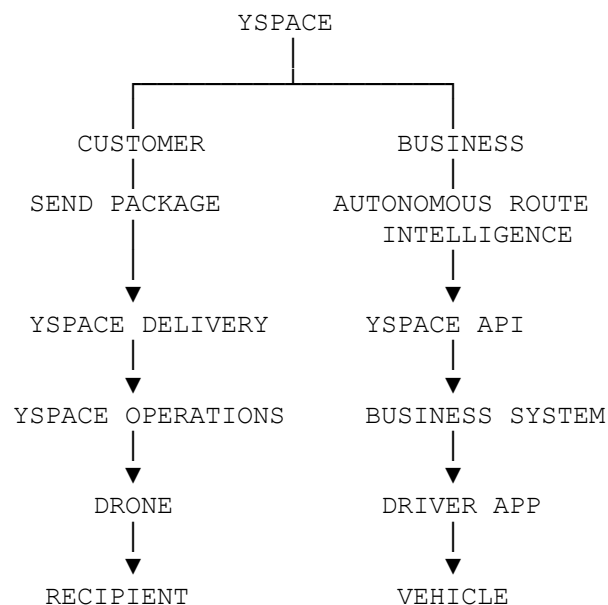
A business that already operates its own transportation network can subscribe to Yspace's intelligence and integrate it into its existing logistics systems.

The business continues to operate its own:

- vehicles;
- riders;
- drivers;
- dispatch;
- fleet-management systems;
- delivery operations.

Yspace provides the intelligence.

166. THE FINAL SYSTEM FLOW



This is the architecture the product team should now design around.

167. UI/UX DESIGNER'S PRIMARY DELIVERABLE

The designer should ultimately produce designs for:

Public Yspace

- Homepage
- Drone Delivery landing page
- Business/Enterprise landing page
- Pricing
- API/integration information
- Support

Customer application

- Sign up
- Sign in
- Home
- Send package
- Location
- Package details
- Estimate
- Checkout
- Confirmation
- Tracking
- History
- Account

Enterprise application

- Enterprise onboarding
- Organization setup
- Dashboard
- Route Intelligence
- Route analysis
- Route results
- Route comparison
- Route details
- Routing profiles
- Route history
- Analytics
- API
- SDK
- Webhooks
- Integrations
- Team
- Billing
- Security
- Settings

168. THE MOST IMPORTANT DESIGN RULE

Do not design the enterprise interface as:

"A better Google Maps."

Design it as:

"A logistics intelligence workspace."

The map is important.

But the map is not the product.

The intelligence surrounding the map is the product.

169. THE MOST IMPORTANT ENGINEERING RULE

Do not build the enterprise product as a replacement fleet-management system.

Build it as:

An intelligence layer that plugs into fleet-management and logistics systems.

170. THE MOST IMPORTANT BUSINESS RULE

Do not sell:

"We manage your riders."

Sell:

"We make the routes your riders take smarter."

171. THE MOST IMPORTANT YSPACEOS RULE

Do not treat Mission Control and the Logistics Platform as the same interface.

They share infrastructure.

They do not share the same user experience.

Mission Control:

Operate Yspace.

Logistics Platform:

Use Yspace.

172. FINAL PRODUCT STATEMENT

Yspace is building an autonomous logistics and transportation infrastructure company powered by ySpaceOS. Yspace Mission Control serves as the internal control tower for Yspace's own autonomous operations, while the Yspace Logistics Platform serves customers and businesses externally. Customers can use Yspace to physically deliver packages through Yspace's drone network. Businesses and logistics enterprises can instead subscribe to Yspace Autonomous Route Intelligence and integrate its routing, ETA and logistics intelligence into their existing fleet and delivery systems, allowing their own riders, drivers and vehicles to operate more efficiently without Yspace managing those assets.

PAYMENT FLOW

1. Direct Yspace Delivery — Individual / Regular Customer

The customer is using Yspace directly to send a package.

The UI flow should be something like:

Website / Mobile App → Create Delivery → Pickup Location → Drop-off Location → Package Information → Package Weight → Package Dimensions → Package Type → Optional Package Photo → Delivery Speed → Price Calculation → Order Summary → Payment → Delivery Confirmation → Live Tracking → Delivery Completed

The pricing engine should automatically calculate the transportation price based on configurable backend rules.

For example:

Package weight: 2 kg

Distance: 8 km

Package category: Standard

Delivery type: Same-day

→ Automatically calculated delivery fee: ₦X

The customer **should never manually calculate the transportation cost**.

The PRD should therefore specify a **Yspace Dynamic Pricing Engine** that can consider:

- Weight
- Dimensions
- Distance
- Pickup/drop-off zones
- Package category
- Delivery urgency
- Drone availability
- Flight/route complexity
- Weather/operational conditions where applicable
- Special handling requirements
- Base transportation fee
- Additional service fees
- Taxes/charges where applicable
- Promotional discounts
- Minimum delivery charge

And the UI should show the customer **how the final price was constructed**, without exposing sensitive internal pricing logic.

2. Marketplace → Yspace API Delivery

This is a completely different customer experience.

The marketplace's customer **doesn't need to know that Yspace exists as a separate delivery app**.

For example:

Marketplace → Customer purchases product → Marketplace already knows product weight/dimensions → Marketplace calls Yspace API → Yspace calculates delivery price → Marketplace displays Yspace delivery option → Customer pays marketplace → Marketplace/Yspace settlement occurs → Yspace receives delivery request → Yspace operates delivery → Customer tracks delivery

This is extremely important.

The marketplace should send Yspace structured information such as:

```
pickup_location
dropoff_location
package_weight
package_dimensions
package_category
order_value
delivery_priority
merchant_information
customer_information
```

Yspace then returns something like:

```
delivery_option
estimated_delivery_time
estimated_distance
delivery_price
currency
service_level
quote_expiry
```

So the marketplace can display:

Yspace Express Delivery

Estimated arrival: Today, 4:30 PM

Delivery fee: ¥X

The marketplace customer pays **through the marketplace's existing checkout**, not necessarily through the Yspace app.

That distinction needs to be explicitly documented in the PRD.

3. Enterprise / Business — Autonomous Route Intelligence

This is the area you clarified, and I agree with your distinction.

Enterprise customers are NOT using the normal Yspace delivery-booking flow.

They are companies that already operate their own logistics fleet.

For example:

Enterprise



Own Fleet Management System



Own Riders / Drivers

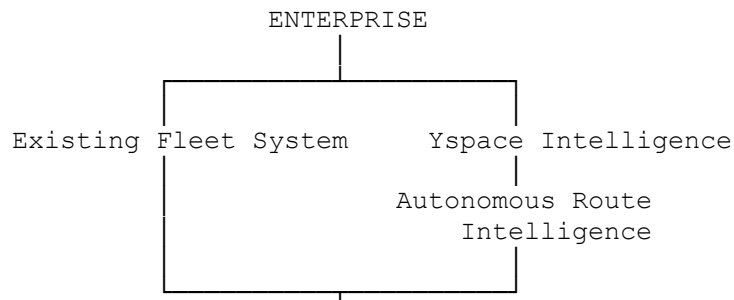


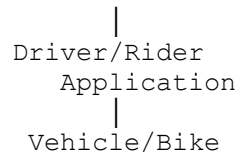
Own Motorcycles / Cars / Vans / Trucks

They subscribe to **Yspace Autonomous Route Intelligence**.

Yspace becomes the intelligence layer sitting alongside or above their existing logistics infrastructure.

The architecture should therefore be:





The enterprise doesn't come to Yspace and say:

“Deliver this package for me.”

Instead, they effectively say:

“Use Yspace intelligence to determine the best route for my fleet.”

That means the enterprise UI needs its **own workspace**.

4. Enterprise Payment Flow

This is where the previous PRD needs significant improvement.

Enterprise payment shouldn't simply be treated like:

Select plan → Pay ~~¥~~X → Done.

Instead, the PRD should support **enterprise commercial arrangements**.

The flow can be:

Yspace Website → Enterprise → Autonomous Route Intelligence → Request Enterprise Access / Contact Sales → Company Information → Fleet Information → Use Case → Fleet Size → Integration Requirements → API/SDK Requirements → Commercial Proposal → Contract → Enterprise Workspace Created → Subscription/Billing Configuration → API Credentials → Integration → Fleet Connected → Intelligence Activated

Depending on the commercial model, Yspace could support:

Enterprise subscription

A company pays a recurring fee for access to the intelligence platform.

Usage-based pricing

The company pays according to things such as:

- Number of vehicles
- Number of route requests
- API calls
- Navigation events
- Monthly active vehicles
- Route calculations
- Geographic coverage
- Data volume

Hybrid enterprise pricing

For example:

Platform fee + per-vehicle fee + usage.

The exact pricing model should **not be hard-coded into the UI/UX PRD yet**. Instead, the PRD should require the billing system to support configurable enterprise pricing.

5. Enterprise Driver/Rider Experience

This is another important thing you just clarified.

The **enterprise owns the drivers/riders and vehicles**.

But those drivers still need a way to receive the intelligence generated by Yspace.

So the PRD should introduce an **Enterprise Driver Interface / Yspace Navigation SDK**.

The flow becomes:

```

Enterprise
  ↓
Yspace Autonomous Route Intelligence
  ↓
Optimal Route Generated
  ↓
Enterprise Fleet System / Yspace SDK/API
  ↓
Driver/Rider Interface
  ↓
Driver/Rider
  ↓
Vehicle

```

The driver does **not** become a Yspace logistics customer.

They are simply consuming route intelligence supplied through their employer's Yspace integration.

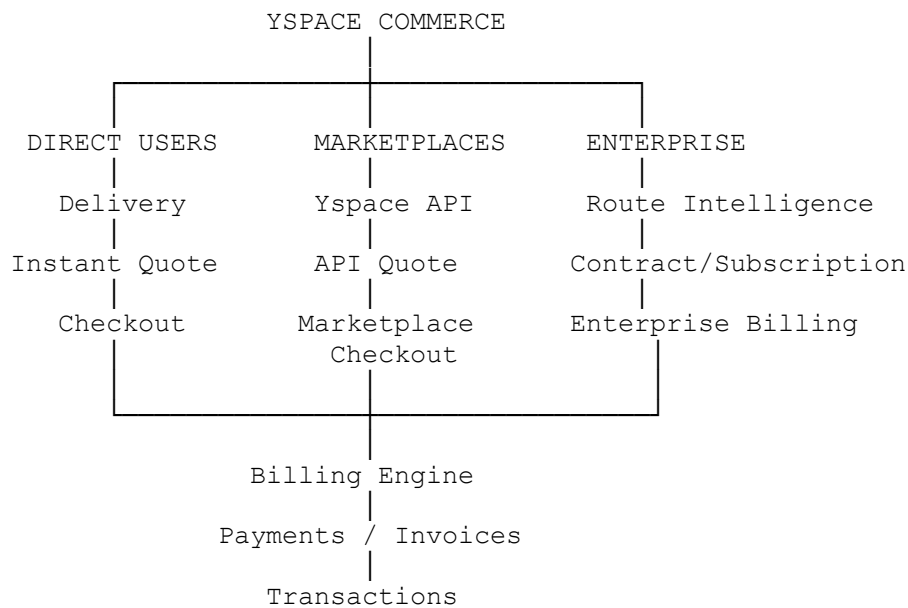
This distinction should be repeated throughout the PRD so the designer and engineering team don't accidentally build the wrong product.

6. One Unified Yspace Billing Architecture

The PRD should therefore introduce a central:

Yspace Commerce & Billing Engine

But with **different billing flows for different customers.**



This is much closer to what the actual product architecture should look like.

7. The PRD numbering should also become much more granular

And yes, I understand what you meant about “2A, 2B, 2C, 2D...”.

Instead of having one vague section called **Payment**, I would structure the long PRD something like:

2. Yspace Logistics Platform

2A. Product Definition

2B. Product Objectives

2C. User Types

2D. Customer Segmentation

2E. Direct Delivery Customer

2F. Marketplace Integration Customer

2G. Enterprise Route Intelligence Customer

2H. Direct Delivery User Journey

2I. Marketplace Delivery Journey

2J. Enterprise Journey

2K. Account Creation

2L. Authentication

2M. Package Creation

2N. Package Information

2O. Weight & Dimension Capture

2P. Location Selection

2Q. Delivery Quote Engine

2R. Dynamic Pricing Engine

2S. Direct Customer Checkout

2T. Payment Processing

2U. Payment Failure

2V. Payment Confirmation

2W. Order Confirmation

2X. Delivery Tracking

2Y. ETA Prediction

2Z. Delivery Completion

Then:

3A. Marketplace Integration

3B. Marketplace API Architecture

3C. Delivery Quote API

3D. Weight & Dimension Data

3E. Delivery Pricing API

3F. Checkout Integration

3G. Order Creation API

3H. Tracking API

3I. Webhooks

3J. API Authentication

3K. API Dashboard

3L. API Usage

3M. API Billing

3N. Marketplace Settlement

And separately:

4A. Enterprise Autonomous Route Intelligence

4B. Enterprise Onboarding

- 4C. Enterprise Account**
- 4D. Enterprise Workspace**
- 4E. Fleet Integration**
- 4F. Vehicle/Rider Data**
- 4G. Route Intelligence**
- 4H. Route Recommendations**
- 4I. Driver/Rider Interface**
- 4J. SDK**
- 4K. Enterprise API**
- 4L. Enterprise Billing**
- 4M. Subscription Management**
- 4N. Usage Billing**
- 4O. Enterprise Invoices**
- 4P. Enterprise Analytics**
- 4Q. Enterprise Administration**

And most importantly: the UI/UX requirements

For **every one of those sections**, the PRD should answer four things:

A. What is the feature?

What does it actually do?

B. Who uses it?

Customer, marketplace, enterprise administrator, driver/rider, Yspace operations team, etc.

C. What does the UI look like?

Exactly what screen, dashboard, modal, card, button, form, map, status indicator, table, notification, etc. is required.

D. What happens technically?

What data enters the system, what backend service processes it, what comes back, and what the user sees.

For example:

Package Weight Screen

Package Information
What does your package weigh?
[2.0] kg
Package dimensions
Length [] cm
Width [] cm
Height [] cm
[Continue]

Then the backend:

```
Weight
+
Dimensions
+
Distance
+
Delivery Type
+
Operational Rules
↓
Dynamic Pricing Engine
↓
Delivery Quote
```

Then the next screen:

DELIVERY SUMMARY	
Pickup	Lagos Island
Destination	Ikeja
Weight	2.0 kg
Distance	XX km

Estimated arrival
Today • 4:30 PM

Delivery fee
~~¥~~X,XXX

[Continue to Payment]